

Sistema de Gestión de Telefonía — Documentación Técnica

Versión del documento: 1.0

Fecha: 11 de septiembre de 2026

Aplicación: velocidad en este repo: `sistema-telefonía`

1. Introducción

1.1 Objetivo del sistema

El **Sistema de Gestión de Telefonía** es una aplicación web para inventariar, administrar y controlar los costos de la telefonía de una organización (fija, móvil, SIMs y equipos). Permite:

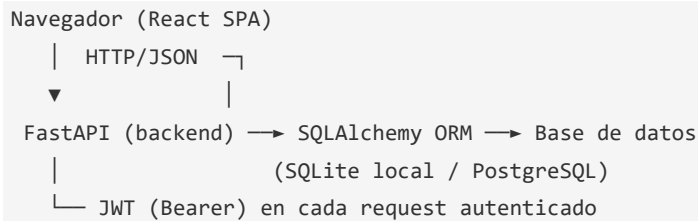
- Registrar la estructura organizativa: personas, departamentos, edificios y locales.
- Inventariar recursos telefónicos: teléfonos fijos, extensiones, líneas móviles, dispositivos y tarjetas SIM.
- Gestionar contratos y planes con información de vencimientos y costos mensuales.
- Asignar recursos a personas y auditar quién tiene cada cosa.
- Cargar costos mensuales y visualizar totales por departamento, operador y período.
- Auditar todos los cambios a través de un historial de movimientos.

1.2 Contexto tecnológico

- **Backend:** Python 3.12, FastAPI 0.115, SQLAlchemy 2.0 (ORM), Alembic (migraciones), Pydantic 2 / Pydantic-Settings, autenticación JWT (python-jose) con contraseñas bcrypt.
 - **Frontend:** React 18 + TypeScript 5, Vite 5, TanStack Query 5, TanStack Table 8, React Router 6, TailwindCSS 3, Recharts (gráficos), Lucide (íconos).
 - **Base de datos:** por defecto PostgreSQL (URL configurable). Para desarrollo local se usa **SQLite** (archivo `backend/test.db`) para simplificar las pruebas.
 - **Patrón de la UI:** paneles de listado con formularios deslizantes (slide-over), validación de formularios centralizada en el frontend, y un componente CRUD genérico reutilizable.
-

2. Arquitectura general

2.1 Vista de alto nivel



El frontend es una **SPA** (single-page application) que se comunica con el backend exclusivamente mediante una **API REST JSON** en el prefijo `/api/v1`. El backend expone todos los datos a través de esa API y aplica autenticación JWT (OAuth2 password flow) en la mayoría de los recursos.

2.2 Separación del monolito por capas (backend)

```

app/
├── main.py           # Crea la app FastAPI, CORS, monta routers
├── core/             # Configuración y seguridad
│   ├── config.py    # Settings desde .env
│   └── security.py  # JWT, bcrypt, dependencias de auth
├── db/              # DeclarativeBase (importa TODOS los modelos)
│   ├── base.py      # engine + SessionLocal + get_db
│   └── session.py
├── models/          # Modelos SQLAlchemy (una tabla por módulo)
├── schemas/         # Schemas Pydantic (Create / Update / Read)
├── services/        # Lógica de negocio reutilizable
├── api/v1/
│   ├── crud.py      # build_crud(): CRUD genérico + historial
│   ├── router.py    # Registra todos los routers
│   └── endpoints/   # Un archivo por módulo/recurso
└── seed.py          # Crea rol admin y usuario admin
  
```

2.3 Separación de capas (frontend)

```

src/
├── main.tsx         # Punto de entrada, providers
├── index.css         # Tailwind + variables de tema
├── app/             # Navegación y rutas
│   ├── nav.ts       # Menú (grupos e ítems, íconos, roles)
│   ├── router.tsx   # Definición de rutas (BrowserRouter)
│   └── ProtectedRoute.tsx # Guards de sesión y de rol admin
├── components/
│   ├── layout/      # AppShell, Sidebar, Topbar
│   ├── ui/          # Button, DataTable, Field, SlideOver,
│   │               # ConfirmDialog, Toast, EmptyState
│   └── crud/        # CrudPage (CRUD genérico por configuración)
├── lib/             # Capa de datos y utilidades
└── api.ts           # Cliente fetch + token + errores ApiError
  
```

	─ queries.ts	# Hooks TanStack Query por recurso
	─ types.ts	# Interfaces TS alineadas con los schemas
	─ validation.ts	# Reglas, limpieza y validación de campos
	─ useFormulario.ts	# Hook de estado de formularios
	─ formatters.ts	# Fechas, moneda, períodos
	─ auth-context.tsx	# Sesión: user, login, logout, isAdmin
	─ query-client.ts	# Cliente global de TanStack Query
	└─ pages/	# Una carpeta por pantalla

2.4 Principios adoptados

1. **CRUD genérico:** una sola implementación (`build_crud` en backend, `CrudPage` en frontend) que se reutiliza para todas las entidades simples.
2. **Tipos alineados:** las interfaces de TypeScript replican 1:1 los schemas Pydantic (los campos que el backend devuelve).
3. **Validación en el frontend:** centralizada en `lib/validation.ts`; los formularios no permiten cargar datos inválidos (solo dígitos en IMEI/ICCID, CUIT con dígito verificador, emails con formato, períodos AAAA-MM, etc.).
4. **Trazabilidad:** cada alta/baja/modificación genera un registro de historial.

3. Backend

3.1 Dependencias y entorno

Archivo `backend/requirements.txt`:

- `fastapi==0.115.6` — framework web y validación con Pydantic.
- `uvicorn[standard]==0.34.0` — servidor ASGI.
- `sqlalchemy==2.0.36` — ORM.
- `psycopg[binary]==3.2.3` — driver PostgreSQL.
- `alembic==1.14.0` — migraciones de base de datos.
- `pydantic==2.10.4` — validación/serialización (incluida con FastAPI).
- `pydantic-settings==2.7.0` — configuración desde variables de entorno.
- `python-jose[cryptography]==3.3.0` — gestión de JWT.
- `bcrypt==4.2.1` — hash de contraseñas.
- `python-multipart==0.0.20` — formularios OAuth2 (/auth/login).

`backend/requirements-dev.txt` agrega `pytest` y `httpx` para pruebas.

Entorno virtual recomendado en `backend/venv` (ya creado en este repo).

3.2 Estructura de carpetas de app/

Carpeta	Función
app/core/	Configuración de la app y seguridad (JWT + contraseñas + dependencias de au
app/db/	Definición de la base declarativa, motor y sesiones.
app/models/	Modelos SQLAlchemy agrupados por dominio: organización, catálogos, person
app/schemas/	Schemas Pydantic en el patrón Base, Create, Update, Read.
app/services/	Lógica de negocio compartida (detalles de línea/teléfono, resúmenes de costos
app/api/v1/endpoints/	Routers FastAPI: uno por recurso; algunos usan el CRUD genérico.
app/api/v1/crud.py	Fábrica build_crud() que genera los cinco verbos (GET listar/uno, POST, PUT, DE
app/api/v1/router.py	Monta todos los routers bajo el prefijo de la API con auth global.

3.3 Configuración (app/core/config.py)

Usa pydantic-settings y lee el archivo .env del directorio de backend (no versionado). Variables:

Variable	Default en código	Descripción
app_name	Sistema de Gestion de Telefonía	Nombre de la app (se muestra en la doc OpenAPI).
api_prefix	/api/v1	Prefijo de la API REST.
database_url	postgresql+psycopg://...	Cadena de conexión a la base de datos.
secret_key	cambiar-en-produccion	Clave para firmar los JWT. Cambiar en producción.
access_token_expire_minutes	30	Minutos de validez del token.
algorithm	HS256	Algoritmo de firma JWT.

.env de desarrollo actual (backend):

```
DATABASE_URL=sqlite:///./test.db
SECRET_KEY=clave-secreta-para-desarrollo
ACCESS_TOKEN_EXPIRE_MINUTES=30
```

3.4 Base de datos y sesiones

- app/db/base.py: define Base(DeclarativeBase). **Importante:** al final del archivo ejecuta `import app.models` para que todos los modelos se registren en `Base.metadata` (necesario para Alembic y para `create_all`). Este import está *después* de la definición de Base para evitar import circular.

- app/db/session.py: crea engine con `pool_pre_ping=True`, `SessionLocal` (sessionmaker) y el generador `get_db()` que usa FastAPI como dependencia para inyectar sesiones y cerrarlas al terminar.

3.5 Modelos de datos

Todos los modelos heredan de Base y usan `Mapped / mapped_column` (SQLAlchemy 2.0). Resumen:

Modelo	Tabla	Campos principales	Relaciones
Rol	roles	nombre (único), descripcion	usuarios

Usuario	usuarios	username (único), email (único), password_hash, activo, rol_id	
Persona	personas	nombre, apellido, documento, email, telefono, departamento_id	
Departamento	departamentos	nombre, departamento_padre_id padre/hijos (jerárquico)	
Edificio	edificios	nombre, direccion	—
Local	locales	edificio_id, piso, oficina, descripcion	edificio
Estado	estados	nombre (único), descripcion	teléfonos, extensiones, SIMs, líneas, dispositivos
Operador	operadores	nombre (único), descripcion	SIMs, líneas, planes
Telefono	telefonos	numero (único), local_id, estado_id, extensiones	local, estado, extensiones
Extension	extensiones	numero, telefono_id, estado_id, operador_id	telefono, estado
Sim	sims	iccid (único), imsi, operador_id, estado	operador, estado
Linea	lineas	numero (único), operador_id, plan_id, local_id, estado_id, dispositivos	operador, plan, local, estado, dispositivos
Dispositivo	dispositivos	marca, modelo, imei (único), linea_id, local_id, estado_id	linea, local, estado
Contrato	contratos	numero, descripcion, fecha_inicio, fecha_vencimiento	linea, fecha
Plan	planes	nombre, operador_id, contrato_id, costo, descripcion	operador, contrato, costo
Asignacion	asignaciones	persona_id, tipo_recurso (linea/dispositivo/extension), recurso_id, fecha_inicio, fecha_fin	persona, recurso, fecha
Coste	costes	periodo (AAAA-MM), concepto, monto, moneda, y departamento_id, linea_id o contrato_id	
Historial	historial	entidad, entidad_id, accion (creado/actualizado/eliminado), campo, valor_anterior, valor_nuevo	

Notas de modelado:

- **Asignaciones polimórficas:** tipo_recurso + recurso_id apuntan a una tabla u otra según el tipo (no hay FK real). El detalle de línea/teléfono usa `services/telefonos.py` para resolver el responsable.
- **Costos polimórficos:** un coste puede imputarse a un departamento, a una línea o a un contrato (los tres opcionales).
- **Departamentos jerárquicos:** departamento_padre_id apunta a la propia tabla (auto-relación con `remote_side=[id]`).

3.6 Schemas Pydantic

Cada entidad define (en `app/schemas/*.py`) el patrón:

- `*Base`: campos comunes (la mayoría opcional si la columna lo permite).
- `*Create` / `*Update`: heredan de `*Base`. `Update` puede usar `passthrough` o campos opcionales; cuando se usa `exclude_unset=True` en el PUT solo se actualizan campos enviados.
- `*Read`: hereda de `*Base`, agrega `id` y `model_config = ConfigDict(from_attributes=True)` para serializar directamente objetos ORM.

Casos particulares:

- `UsuarioRead` devuelve el objeto rol **anidado** (`rol: RolRead`).
- `Token` (`access_token`, `token_type="bearer"`) para la respuesta de login.
- Los schemas de detalle (`LineaDetalle`, `TelefonoDetalle`, `ContratoDetalle`, `PlanResumen`, `ExtensionResumen`) son armados a mano en los endpoints/services.

3.7 CRUD genérico (app/api/v1/crud.py)

La función `build_crud(router, model, Create, Update, Read, entidad=None)` registra en el router:

- `GET /` → lista con `response_model=list[Read]`, paginación `skip/limit` (limit max 500).
- `GET /{id}` → un elemento o 404.
- `POST /` → crea (201) y registra historial creado.
- `PUT /{id}` → actualiza con `exclude_unset=True`, registra historial por cada campo cambiado (campo, valor_anterior, valor_nuevo).
- `DELETE /{id}` → elimina (204) y registra historial eliminado.

La operación: `model(**payload.model_dump())` para crear, `setattr` para actualizar. El historial usa `request.state.user` (seteado por `get_current_user`) para el `usuario_id`.

Siempre que una entidad pase `entidad`, los cambios quedan trazados en el historial.

3.8 Endpoints por módulo

Referencia completa en la sección 7. Routers principales: `auth` (`login/me`), `usuarios`, `roles`, y los generados por `build_crud` para: `personas`, `departamentos`, `edificios`, `locales`, `estados`, `operadores`, `teléfonos`, `extensiones`, `líneas`, `dispositivos`, `SIMs`, `planes`, `contratos`, `costos`, `asignaciones`, `historial` y `reportes`.

Endpoints custom (además del CRUD):

- `POST /auth/login`, `GET /auth/me`.
- `GET /lineas/{id}/detalle`, `GET /telefonos/{id}/detalle`, `GET /contratos/{id}/detalle`.
- `GET /extensiones/buscar?q=`, `GET /extensiones/por-telefono/{id}`.
- `GET /locales/por-edificio/{id}`.
- `GET /asignaciones/activas`, `/asignaciones/por-recurso?tipo_recurso=&recurso_id=`, `/asignaciones/por-persona/{id}`, `PUT /asignaciones/{id}/finalizar`.
- `GET /historial/` con filtros `entidad` y `entidad_id`.
- `GET /reportes/inventario`, `/reportes/costes-totales`, `/reportes/costes-por-departamento`, `/reportes/costes-por-operador`, `/reportes/costes-por-periodo`, `/reportes/recursos-por-departamento`.
- `GET /health` (fuera del prefijo) para chequeos.

3.9 Autenticación y seguridad

- `app/core/security.py`:
 - `hash_password / verify_password`: `bcrypt`.
 - `create_access_token(subject)`: JWT HS256 con `sub = username` y `exp = ahora + access_token_expire_minutes`.
 - `get_current_user`: dependencia que decodifica el Bearer token, busca el usuario y guarda `request.state.user`.

- `require_role(*roles)`: devuelve una dependencia que lanza 403 si el rol del usuario no está en la lista.
- Flujo de login: POST `/auth/login` recibe `username+password` como `form-urlencoded` (estándar OAuth2), verifica contraseña y activo, devuelve Token. El frontend guarda el token en `localStorage` (clave `troncal.token`).
- Protección de rutas: en `router.py` todos los routers (excepto `auth`) se montan con `dependencies=[Depends(get_current_user)]`. Solo usuarios y roles exigen rol `admin` (`require_role("admin")`).
- Rol `admin`: acceso total. Los roles `gestor` y `consulta` están definidos en la migración inicial, pero en el backend actual solo `admin` tiene restricciones adicionales (el resto de recursos está abierto a cualquier usuario autenticado). Esto queda documentado para saber dónde aplicar permisos por rol si se necesita.

3.10 Servicios (app/services/)

- `telefonía.py`:
 - `get_responsable_id(tipo, recurso_id)`: devuelve la persona asignada activa (fecha_fin nula), priorizando la asignación más reciente.
 - `get_telefono_detalle(...)`: número, ubicación (local/edificio), estado, observaciones y lista de extensiones con responsable.
 - `get_linea_detalle(...)`: número, operador, plan, SIM (iccid/imsi), estado, dispositivo asignado y responsable.
- `costes.py`: totales por período/departamento/línea/contrato y resúmenes agregados por departamento y por período (suma de monto + cantidad de registros).

3.11 Seed (app/seed.py)

Crea (si no existen) el rol `admin` y el usuario `admin` con contraseña `admin123`. Uso en local:

```
python -m app.seed # o: python app/seed.py (desde backend/)
```

3.12 Migraciones (Alembic)

- Configuración: `alembic/env.py` lee `settings.database_url` y usa `Base.metadata` como metadatos.
- Migraciones existentes en `backend/alembic/versions/`:
 - `001_initial.py`: crea las 18 tablas e inserta los roles `admin`, `gestor`, `consulta`.
 - `002_quitar_proveedores.py`: elimina la tabla `proveedores` y la columna `contratos.proveedor_id`.

Comandos útiles (desde `backend/` con el `venv` activo):

```
alembic upgrade head
alembic revision --autogenerate -m "descripcion"
alembic current | history
```

Para que `--autogenerate` detecte cambios del modelo hay que tener los `import` registrados en `app/models/__init__.py`.

3.13 CORS

app/main.py configura CORS abierto (allow_origins=["*"]) apto para desarrollo. En producción se debe restringir al dominio del frontend.

4. Frontend

4.1 Dependencias y scripts (frontend/package.json)

Scripts:

Comando	Función
npm run dev	Servidor de desarrollo Vite (por defecto puerto 5173).
npm run typecheck	tsc --noEmit.
npm run build	tsc --noEmit && vite build (genera dist/).
npm run preview	Sirve el build de producción.

Dependencias destacadas: react, react-dom, react-router-dom, @tanstack/react-query, @tanstack/react-table, recharts, lucide-react, tailwindcss, typescript, vite, zod y react-hook-form (instalados pero **no usados**: los formularios usan el sistema propio useFormulario).

4.2 Configuración del entorno (frontend/.env)

```
VITE_API_URL=http://localhost:8000/api/v1
```

Si falta, lib/api.ts usa como default http://localhost:8000/api/v1.

4.3 Arranque de la app (src/main.tsx)

Orden de providers:

1. QueryClientProvider (TanStack Query) con queryClient global de lib/query-client.ts.
2. ToastProvider (notificaciones UI).
3. AuthProvider (sesión).
4. AppRouter (routing con BrowserRouter).

4.4 Capa de datos

- lib/api.ts:
 - api<T>(path, opts) — wrapper de fetch: agrega el header Authorization si hay token, maneja JSON o form, dispara logout automático en 401 (salvo en /auth/login) y convierte errores a ApiError (status + detail). Los errores de validación de Pydantic (array detail) se convierten a un mensaje unido por .
 - tokenStore — acceso a localStorage clave troncal.token.

- `qs(params)` — serializa query params sin claves vacías.
- `lib/queries.ts`: hooks por recurso usando `listado<T>(key, path)` (ej. `usePersonas`, `useLineas`). Los `queryKey` son `[key, "list"]`. Custom queries para detalles, asignaciones (activas/por persona/por recurso), historial y reportes. `useCrudMutations<T>` crea `crear/actualizar/eliminar` con invalidación automática de la caché.
- `lib/types.ts`: interfaces TS alineadas 1:1 con los schemas Pydantic (misma convención de nombres: `LineaDetalle`, `ContratoDetalle`, etc.).
- `lib/query-client.ts`: instancia de `QueryClient` con defaults razonables.

Regla de oro: no declarar en `types.ts` campos que el backend no devuelva; si se agrega un campo al backend, agregarlo también aquí.

4.5 Sesión (lib/auth-context.tsx)

- En cada arranque, si hay token, carga `/auth/me` y guarda el usuario en estado.
- `login(username, password) → POST /auth/login (form)`, guarda token, carga `/auth/me`.
- `logout()` → limpia token y redirige a `/login`.
- `isAdmin = user.rol.nombre === "admin"`.
- Guards: `ProtectedRoute` (sin sesión → `/login`) y `AdminRoute` (no admin → `/`).

4.6 Validación de formularios (core: lib/validation.ts + lib/useFormulario.ts)

TipoCampo:

Tipo	Regla
text	opcional/obligatorio, min/max caracteres
textarea	igual que text (justifica arriba)
select	solo se exige obligatorio (valores vienen de opciones)
password	mín 6 caracteres
numero	solo dígitos (IMEI, ICCID, documento, IMSI), reglas de longitud
telefono	dígitos, espacios, guiones, +; mín 6 dígitos
decimal	número con coma o punto (monto, coste mensual)
email	formato básico <code>a@b.c</code>
date	fecha AAAA-MM-DD real (valida días del mes)
periodo	AAAA-MM con mes 01–12 (costos)
cuit	11 dígitos + dígito verificador (módulo 11)

`ReglaCampo: { name, label, tipo, required?, min?, max? }.`

Funciones:

- `limpiar(tipo, raw)`: sanitiza al escribir (elimina letras en campos numéricos, normaliza coma→punto en decimales, quita espacios en email, etc.). Se usa en `setValor`.
- `validarCampo(valor, regla)`: devuelve mensaje de error o `null`.
- `verificarCuit(cuit)`: dígito verificador AFIP.

- `toPayload(reglas, valores)`: arma el payload a enviar — convierte selects a Number, decimales con coma→punto, vacíos a null, textos opcionales vacíos a null.

Hook `useFormulario(campos)`:

Método	Descripción
<code>valores / errores</code>	estado del formulario
<code>setValor(name, raw)</code>	limpia según el tipo y borra el error del campo
<code>setValores(item)</code>	llena el formulario desde un registro (null lo vacía)
<code>validarUno(name)</code>	valida un campo (típicamente en blur)
<code>validarTodos()</code>	valida todo; devuelve bool
<code>marcarError(name, msg)</code>	setea/limpia un error programáticamente (validación cruzada, ej. fechas de con

4.7 Componente CRUD genérico (components/crud/CrudPage.tsx)

`CrudPage<T>` recibe:

- `titulo`, `descripcionVacio`, `entidadKey`, `basePath`.
- `useLista`: hook de query.
- `columnas`: definiciones de `ColumnDef` para `DataTable`.
- `campos`: `CampoConfig[]` — configuración de formulario:

```
type CampoConfig =
  | { name; label; required?; placeholder?; min?; max?; type: TipoCampoTexto; dato? }
  | { ... type: "textarea" }
  | { ... type: "select"; options?: {value;label}[] }
```

- `nombreItem(item)`: para el mensaje de confirmación de borrado.
- `accionesExtra`: botones opcionales.

Internamente: renderiza el listado (`DataTable`), un formulario en `SlideOver` con los campos generados según su tipo, validación con `useFormulario`, y `ConfirmDialog` para borrar. Esto hace que agregar una pantalla CRUD sea declarar columnas + campos.

4.8 Componentes UI (components/ui/)

- `Field.tsx`: `FieldWrapper` (label + children + hint/error) y `TextField`, `TextAreaField`, `SelectField` (con opción "Sin asignar"). El prop `dato` aplica estilo de tachado con la clase CSS `.dato`.

- `DataTable.tsx`: tabla TanStack Table con columnas configurables, filtro de texto, paginado de a 10 y acciones Ver/Editar.

- `SlideOver.tsx`: panel lateral deslizante (usado para formularios y para ver detalles).
- `ConfirmDialog.tsx`: diálogo de confirmación de borrado.
- `Toast.tsx`: notificaciones breves (éxito/error) con `useToast()`.
- `EmptyState.tsx` / `ErrorState.tsx`: estados de carga/vacío/error.

- `Button.tsx`: botones con variantes (primario, destructivo, texto, etc.).

4.9 Páginas

Páginas que usan el **CRUD genérico** (mínima configuración): estados, operadores, extensiones, SIMs, dispositivos y planes.

Páginas **personalizadas** (con `useFormulario` y lógica propia): personas, departamentos, locales (con dos formularios: local y edificio), teléfonos, líneas, contratos (con detalle y validación cruzada de fechas), costes (con filtro por período), asignaciones (dinámicas según tipo de recurso + botón "finalizar"), usuarios (rol admin) e historial (solo lectura con filtro). A estas se suman login y el dashboard.

4.10 Navegación y protección

- `app/nav.ts` define grupos del menú (Directorio, Recursos, Contratos, Catálogos, etc.) e ítems con ícono Lucide. El ítem Usuarios tiene `soloAdmin: true`.
- `app/router.tsx` mapea cada ruta a su página; todas dentro de `<ProtectedRoute>`. `/usuarios` adicionalmente en `<AdminRoute>`.

4.11 Formateadores (`lib/formatters.ts`)

- `formatFecha` (dd/mm/aaaa es-AR), `formatFechaHora`, `formatMoneda` (ARS → \$ x.xxx,xx), `periodoActual` (AAAA-MM), `diasHasta` (días restantes para una fecha).

4.12 Estilos

TailwindCSS 3 con `index.css` que define las clases utilitarias personalizadas del tema (colores como `tinta`, `papel`, `senal`, `linea-baja`, etc.) en `:root/@layer`. Tipografía: sistema, con las clases de diseño del kit.

5. Base de datos

El esquema actual (post eliminar proveedores) tiene **18 tablas**. Diagrama resumido de relaciones:

```
roles 1-n usuarios
departamentos 1-n personas
departamentos 1-n departamentos (jerarquía)
edificios 1-n locales
estados      1-n telefonos / extensiones / sims / lineas / dispositivos
operadores  1-n sims / lineas / planes
locales      1-n telefonos
telefonos    1-n extensiones
sims         1-1 lineas
planes       1-n lineas
contratos    1-n planes
```

```

personas      1-n asignaciones (polimórfica a linea/dispositivo/extension)
departamento/linea/contrato 1-n costes (opcional según imputación)
usuarios      1-n historial

```

Detalle de tablas y columnas en la migración `001_initial.py` y en los modelos `app/models/*.py`.

6. API REST — Referencia completa

Prefijo base: `/api/v1`. Todos los endpoints requieren header `Authorization: Bearer <token>` salvo `/auth/login` y `/health`.

Códigos de respuesta: 200 OK, 201 creado, 204 sin contenido (DELETE), 401 sin autenticar, 403 sin permiso de rol, 404 no existe, 400/422 errores de validación. En errores la respuesta es `{"detail": "mensaje"}` (o array de errores de validación).

Método	Ruta	Descripción
GET	<code>/health</code>	Health check (sin auth)
POST	<code>/auth/login</code>	Login OAuth2 (form) → <code>{access_token}</code>
GET	<code>/auth/me</code>	Usuario actual (con rol anidado)
GET/POST	<code>/usuarios/</code>	Listar / crear usuario (admin)
GET/PUT/DELETE	<code>/usuarios/{id}</code>	Ver / actualizar / eliminar usuario (admin)
GET/POST	<code>/roles/</code>	Listar / crear rol (admin)
GET/PUT/DELETE	<code>/roles/{id}</code>	CRUD rol (admin)
GET/POST	<code>/personas/</code>	CRUD personas
GET/PUT/DELETE	<code>/personas/{id}</code>	CRUD persona
GET/POST	<code>/departamentos/</code>	CRUD departamentos
GET/PUT/DELETE	<code>/departamentos/{id}</code>	CRUD departamento
GET/POST	<code>/edificios/</code>	CRUD edificios
GET/PUT/DELETE	<code>/edificios/{id}</code>	CRUD edificio
GET/POST	<code>/locales/</code>	CRUD locales
GET/PUT/DELETE	<code>/locales/{id}</code>	CRUD local
GET	<code>/locales/por-edificio/{edificio_id}</code>	Locales de un edificio
GET/POST	<code>/estados/</code>	CRUD estados
GET/PUT/DELETE	<code>/estados/{id}</code>	CRUD estado
GET/POST	<code>/operadores/</code>	CRUD operadores
GET/PUT/DELETE	<code>/operadores/{id}</code>	CRUD operador
GET/POST	<code>/telefonos/</code>	CRUD teléfonos
GET/PUT/DELETE	<code>/telefonos/{id}</code>	CRUD teléfono
GET	<code>/telefonos/{id}/detalle</code>	Detalle con extensiones y responsable
GET/POST	<code>/extensiones/</code>	CRUD extensiones
GET/PUT/DELETE	<code>/extensiones/{id}</code>	CRUD extensión
GET	<code>/extensiones/buscar?q=</code>	Búsqueda por número

GET	/extensiones/por-telefono/{id}	Extensiones de un teléfono
GET/POST	/lineas/	CRUD líneas
GET/PUT/DELETE	/lineas/{id}	CRUD línea
GET	/lineas/{id}/detalle	Detalle con operador, plan, SIM, dispositivo y responsable
GET/POST	/dispositivos/	CRUD dispositivos
GET/PUT/DELETE	/dispositivos/{id}	CRUD dispositivo
GET/POST	/sims/	CRUD SIMs
GET/PUT/DELETE	/sims/{id}	CRUD SIM
GET/POST	/planes/	CRUD planes
GET/PUT/DELETE	/planes/{id}	CRUD plan
GET/POST	/contratos/	CRUD contratos
GET/PUT/DELETE	/contratos/{id}	CRUD contrato
GET	/contratos/{id}/detalle	Detalle con planes
GET/POST	/asignaciones/	CRUD asignaciones
GET/PUT/DELETE	/asignaciones/{id}	CRUD asignación
GET	/asignaciones/activas	Asignaciones sin fecha de fin
GET	/asignaciones/por-recurso?tipo_recurso={recurso_id}	Asignaciones de un recurso
GET	/asignaciones/por-persona/{persona_id}	Asignaciones de una persona
PUT	/asignaciones/{id}/finalizar	Pone fecha_fin = hoy
GET/POST	/costes/	CRUD costos
GET/PUT/DELETE	/costes/{id}	CRUD costo
GET	/historial/	Registros (filtros entidad, entidad_id)
GET	/reportes/inventario	Contadores (líneas, teléfonos, dispositivos, personas, edificios)
GET	/reportes/costes-totales	Monto total y cantidad de períodos
GET	/reportes/costes-por-departamento	Suma y cantidad por departamento
GET	/reportes/costes-por-operador	Suma y cantidad por operador (vía línea)
GET	/reportes/costes-por-periodo	Suma y cantidad por período
GET	/reportes/recursos-por-departamento	Líneas y dispositivos asignados por departamento

La documentación interactiva (Swagger) queda en <http://localhost:8000/docs> al correr el backend.

7. Guía de instalación y puesta en marcha

7.1 Backend

```
cd backend
py -m venv venv # si no existe
.\venv\Scripts\activate
pip install -r requirements.txt # y -r requirements-dev.txt si se testea
# crear .env con: DATABASE_URL / SECRET_KEY / ACCESS_TOKEN_EXPIRE_MINUTES
alembic upgrade head # crea/aplica las tablas
```

```
python app/seed.py          # crea rol admin y usuario admin/admin123
uvicorn app.main:app --reload # http://localhost:8000
```

7.2 Frontend

```
cd frontend
npm install
# crear .env con VITE_API_URL=http://localhost:8000/api/v1 (si no existe)
npm run dev          # http://localhost:5173
```

7.3 Ingreso inicial

- Usuario: admin / Contraseña: admin123 (creado por el seed). Cambiar luego desde Administración → Usuarios.

8. Cómo modificar / agregar funcionalidades

8.1 Agregar una entidad simple (recomendado: usar el CRUD genérico)

Ejemplo: agregar entity "categoría" de equipos.

1. **Modelo:** agregar class `Categoria(Base)` en un archivo de `app/models/`, y registrarlo en `app/models/__init__.py`.

2. **Schema:** crear `app/schemas/categorias.py` CON `CategoriaBase/Create/Update/Read`.

3. **Endpoint:** crear `app/api/v1/endpoints/categorias.py`:

```
router = APIRouter(prefix="/categorias", tags=["categorias"])
build_crud(router,      Categoria,      CategoriaCreate,      CategoriaUpdate,      CategoriaRead,
entidad="categorias")
```

4. **Router:** importar y montar el router en `app/api/v1/router.py` (con `dependencies=_auth`).
5. **Migración:** `alembic revision --autogenerate -m "agrega categorias"` y revisar que el SQL sea correcto; luego `alembic upgrade head`.
6. **Frontend types:** agregar la interfaz en `lib/types.ts`.
7. **Frontend queries:** `export const useCategorias = listado<Categoria>("categorias", "/categorias/");`.
8. **Frontend página:** crear `src/pages/categorias/CategoriasPage.tsx` CON `<CrudPage<Categoria> ... campos=[...] columnas=[...]/>`.
9. **Nav y rutas:** agregar al NAV y a `router.tsx`.

8.2 Agregar un endpoint custom

Agregar funciones decoradas en el archivo del módulo correspondiente (ej. `extensiones.py` ya tiene `buscar` y `por-telef`

ono). Recordar:

- Si el path contiene {id} debe definirse **antes** del verbo GET genérico o evitar colisión con /extensiones/{id}.

En FastAPI el orden importa.

- Registrar la lógica de negocio en app/services/*.py si se reutiliza.

8.3 Agregar un tipo de campo de validación nuevo

1. Agregar el valor a TipoCampo en lib/validation.ts.
2. Implementar el caso en limpiar() (sanitización al escribir) y en validarCampo() (regla de validez).
3. Si es un campo de texto con input especial, mapearlo en inputAttrs() de CrudPage.tsx.
4. Documentar el tipo en la tabla de la sección 4.6.

8.4 Cambiar a PostgreSQL en producción

- Instalar el driver (ya está en requirements.txt).
- Configurar DATABASE_URL=postgresql+psycopg://usuario:password@host:puerto/telefonía.
- Correr alembic upgrade head y luego python app/seed.py.
- En app/main.py, restringir allow_origins al dominio del frontend.
- Cambiar SECRET_KEY por un valor seguro.

8.5 Buenas prácticas

- Mantener types.ts sincronizado con los schemas Pydantic.
- Nuevos modelos: registrarlos en app/models/__init__.py (base.py los importa).
- Validar en el frontend con ReglaCampo; no duplicar reglas por página.
- Usar CrudPage cuando no haya lógica custom; reservar páginas manuales para casos específicos.
- Correr npm run typecheck y npm run build antes de terminar.
- No versionar .env ni test.db.

9. Verificación y pruebas

- Backend corriendo: http://localhost:8000/health → {"status":"ok"}.
- Docs interactivos: http://localhost:8000/docs.
- Login por consola (curl):

```
curl -X POST http://localhost:8000/api/v1/auth/login -H "Content-Type: application/x-www-form-urlencoded" -d "username=admin&password=admin123"
```

- Frente frontend: npm run typecheck y npm run build.

- Smoke test rápido de la API (ej. crear y borrar un contrato) usando `TestClient` de FastAPI (ver `backend/requirements-dev.txt`).

10. Problemas comunes y soluciones

Problema	Causa / Solución
No module named 'app' al correr un script	Correr desde backend/ o setear PYTHONPATH al directorio de app.
"Import circular" en modelos	El import de <code>app.models</code> va al final de <code>db/base.py</code> , después de <code>class Base</code> . No m
Alembic no detecta nuevos modelos	Falta importarlos en <code>app/models/__init__.py</code> .
alembic upgrade falla sobre SQLite con DROP COLUMN	Usar <code>op.batch_alter_table(...)</code> como en <code>002_quitar_proveedores.py</code> .
401 después del deploy	Token vencido (30 min) o SECRET_KEY distinta al firmarla.
CORS en producción	Ajustar <code>allow_origins</code> en <code>main.py</code> .
Login "Usuario o contraseña incorrectos"	Verificar <code>seed.py</code> y que <code>bcrypt.hashpw</code> se aplique al crear usuarios.
El campo select en edición queda en blanco	El valor viene como int del backend; <code>useFormulario</code> lo convierte a string con Str

11. Configuración de referencia

backend/.env

```

DATABASE_URL=sqlite:///./test.db
SECRET_KEY=clave-secreta-para-desarrollo
ACCESS_TOKEN_EXPIRE_MINUTES=30

```

frontend/.env

```

VITE_API_URL=http://localhost:8000/api/v1

```

12. Historial de cambios relevantes

- **002_quitar_proveedores:** se eliminó el módulo "Proveedores" (tabla, endpoints, página, menú y campo en contratos) — 11/09/2026.
- **Validación de formularios:** se creó `lib/validation.ts` y `lib/useFormulario.ts`, se migraron todas las páginas para no aceptar datos inválidos — 11/09/2026.
- **Configuración de desarrollo:** base SQLite en `.env`, import circular de `base.py` corregido, seed de admin —

10-11/09/2026.